ST7071CEM: Intelligent Information Retrieval

Submitted To

Name: Siddhartha Neupane

Submitted by

Name: Aishwarya Rai
Student Id: **250061**
CoventryUniversity Student Id: **16542402**

Table Of Contents

Task 1: Vertical Search Engine

1.1. Project Overview

This project develops a dedicated vertical search engine tailored to discover academic publications—both papers and books—authored by members of Coventry University's School of Economics, Finance, and Accounting. It delivers a full end-to-end solution that encompasses data collection, processing, indexing, and search capabilities, functioning as a focused alternative to Google Scholar for this specific academic domain.

The system's key features include:

- **Automated Web Crawling:** A reliable and respectful crawler scans the university portal to collect publication details.

- **Regular Updates:** The crawler runs weekly to capture newly added publications and ensure the index remains up to date.

- **Efficient Indexing:** Collected text is cleaned, processed, and organized into an inverted index and TF-IDF model to enable quick searches and relevance scoring. These indexes are created on the publication title and the author's profile url. Since the author's profile url is unique, this gives more accurate results when user searches based on the author.

- **Relevance-Driven Search:** Through a user-friendly web interface, queries are matched against the index, returning the most relevant publications ranked using cosine similarity.

The entire pipeline is implemented in Python, making use of powerful libraries such as **Selenium** for dynamic scraping, **NLTK** for natural language processing, and **Flask** for building the web application.

1.2. System Architecture

1. The search engine is designed with a modular architecture, where each component has a distinct responsibility. The flow of data and processes can be visualized as follows:
2. Crawler (crawler.py): Initiated either manually or by a scheduler, it navigates the target website, respects its robots.txt rules, and extracts raw publication data (title, authors, links, etc.).
3. File Handler (fileHandler.py): The raw data from the crawler is passed to the main application, which uses this utility module to save it to disk.
4. Indexer (indexer.py): The main application triggers the indexer to process the raw data. The indexer performs two key tasks:
    a. It builds an inverted index for fast document lookup.

   b. It creates a TF-IDF matrix and vocabulary for calculating search result relevance.
   c. These data structures are then saved to disk for persistence.
5. Main App / Web Server (main.py):
   a. This component, built with Flask, serves the user interface.
   b. It manages a background scheduler (apscheduler) to automate the crawling and indexing process weekly.
   c. When a user submits a query, it orchestrates the search.
6. Search (search.py):
   a. The main app passes the user's query and the loaded indexes to the search module.
   b. The search module processes the query, uses the indexes to find and rank relevant documents, and returns a sorted list of results to the main app.
7. User Interface: The main app renders the final, ranked results for the user.

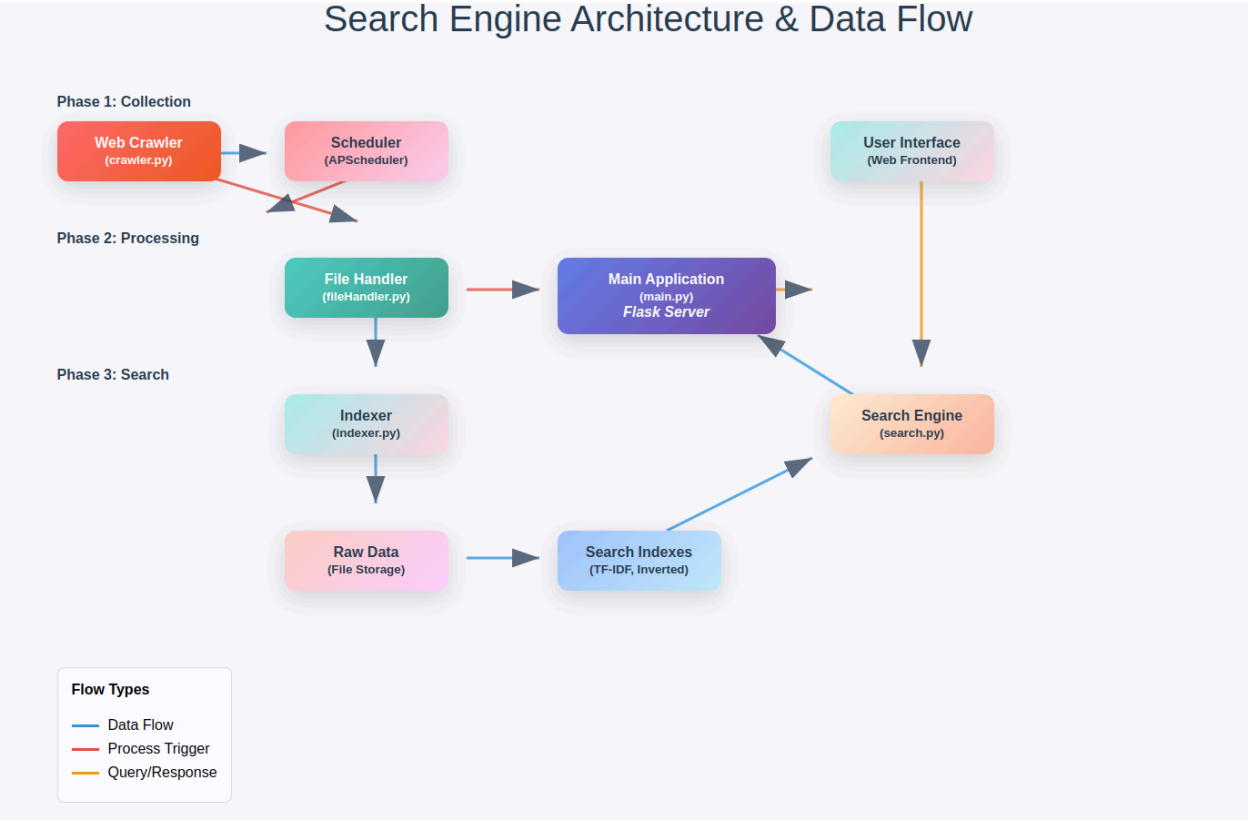


Figure 1: Flow diagram of system architecture

1.3. Module Description

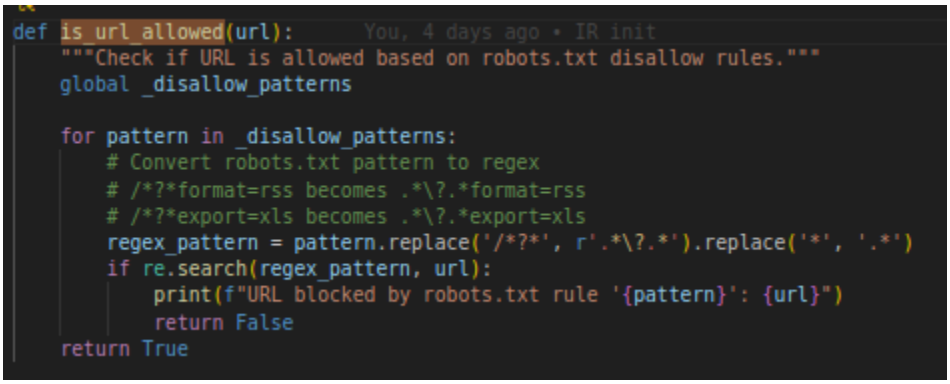
Module 1: Web Crawler (crawler.py)

1. Ethical Crawling
    - robots.txt Compliance: get_robot_rules_and_sitemap() parses crawl delays and disallow rules

```python
def get_robot_rules_and_sitemap():
    """Fetches robots.txt using Selenium and parses disallow rules."""
    global _crawl_delay, _disallow_patterns

    print("Fetching robots.txt with Selenium...")
    driver = get_chrome_driver()  # Use shared driver

    try:
        print(f"Loading robots.txt with Selenium...{ROBOTS_URL}")
        driver.get(ROBOTS_URL)
        time.sleep(10)

        # Get text content directly from page_source using regex instead of BeautifulSoup
        content = driver.page_source
        # Extract text content between <pre> tags if present, otherwise use full text
        text_match = re.search(r'<pre[^>]*>(.*?)</pre>', content, re.DOTALL)
        if text_match:
            text_content = text_match.group(1)
            # Remove HTML entities
            text_content = text_content.replace('&lt;', '<').replace('&gt;', '>').replace('&amp;', '&')
        else:
            # Fallback: strip all HTML tags using regex
            text_content = re.sub(r'<[^>]+>', '', content)

        print("Successfully fetched robots.txt with Selenium!")
        print("Content preview:", text_content[:200])

        # Parse sitemaps, crawl delay, and disallow rules
        sitemaps = []
        crawl_delay = 5
        disallow_patterns = []

        for line in text_content.splitlines():
            line = line.strip()
            if re.match(r'^sitemap:', line, re.IGNORECASE):
                sitemap_url = line.split(':', 1)[1].strip()
                sitemaps.append(sitemap_url)
                print(f"Found sitemap: {sitemap_url}")
            elif re.match(r'^crawl-delay:', line, re.IGNORECASE):
                crawl_delay = int(line.split(':', 1)[1].strip())
                print(f"Found crawl delay: {crawl_delay}")
            elif re.match(r'^disallow:', line, re.IGNORECASE):
                disallow_path = line.split(':', 1)[1].strip()
                disallow_patterns.append(disallow_path)
                print(f"Found disallow rule: {disallow_path}")

        _crawl_delay = crawl_delay
        _disallow_patterns = disallow_patterns
        print(f"Loaded {len(disallow_patterns)} disallow rules from robots.txt")

        return crawl_delay, sitemaps[0] if sitemaps else None

    except Exception as e:
        print(f"Selenium failed: {e}")
        return 5, None
```

Fig 2: get_robot_rules_and_sitemap() code implementation

- URL Validation: is_url_allowed() verifies every URL against restrictions before access



```python
def is_url_allowed(url):      You, 4 days ago • IR init
    """Check if URL is allowed based on robots.txt disallow rules."""
    global _disallow_patterns

    for pattern in _disallow_patterns:
        # Convert robots.txt pattern to regex
        # /*?*format=rss becomes .*\?.*format=rss
        # /*?*export=xls becomes .*\?.*export=xls
        regex_pattern = pattern.replace('/*?*', r'.*\?.*').replace('*', '.*')
        if re.search(regex_pattern, url):
            print(f"URL blocked by robots.txt rule '{pattern}': {url}")
            return False
    return True
```

Fig 3: is_url_allowed() code implementation

2. Crawling Workflow
    - Driver Initialization: Single persistent Chrome instance reduces overhead
    - Page Discovery: Direct access to publications page with sitemap fallback
    - Data Extraction:
        - scrape_publication_data() extracts titles, years, and links from paginated results

```python
def scrape_publication_data(org_url, driver, author_hrefs):
    """Scrapes publication details using Selenium directly instead of BeautifulSoup."""
    page_publications = []

    try:
        # Wait for page to load completely
        WebDriverWait(driver, 15).until(
            EC.presence_of_element_located((By.CSS_SELECTOR, 'div.result-container'))
        )

        # Get count of publication containers first
        result_containers = driver.find_elements(By.CSS_SELECTOR, 'div.result-container')
        total_results = len(result_containers)
        print(f"Found {total_results} publication containers on page")

        # Process each publication by index to avoid stale element references
        for index in range(total_results):
            try:
                # Re-find elements each time to avoid stale references
                current_results = driver.find_elements(By.CSS_SELECTOR, 'div.result-container')

                if index >= len(current_results):
                    print(f"Skipping index {index} - container no longer exists")
                    continue

                current_item = current_results[index]

                # Extract data from current item
                title = "No Title"
                pub_link = "#"
                year = "N/A"

                # Get title
                try:
                    title_elements = current_item.find_elements(By.CSS_SELECTOR, 'h3.title')
                    if title_elements:
                        title = title_elements[0].text.strip()

                        # Get publication link
                        link_elements = title_elements[0].find_elements(By.TAG_NAME, 'a')
                        if link_elements:
                            href = link_elements[0].get_attribute('href')
                            if href:
                                pub_link = urljoin(BASE_URL, href)
                except Exception as e:
                    print(f"Error extracting title for item {index}: {e}")

                # Get year
                try:
                    year_elements = current_item.find_elements(By.CSS_SELECTOR, 'span.date')
                    if year_elements:
                        year = year_elements[0].text.strip()
                except Exception as e:
                    print(f"Error extracting year for item {index}: {e}")

                # Fetch authors from the detail page (this navigates away from current page)
                authors = []
                if pub_link != "#":
                    authors = scrape_author_info(pub_link, driver, author_hrefs)

                    # Navigate back to the original page after scraping author info
                    driver.get(org_url)
                    # Wait for the page to reload
                    WebDriverWait(driver, 10).until(
                        EC.presence_of_element_located((By.CSS_SELECTOR, 'div.result-container'))
                    )

                page_publications.append({
                    "title": title,
                    "authors": authors,
                    "year": year,
                    "publication_url": pub_link,
                    "org_url": org_url
                })

                print(f"✓ Processed publication {index + 1}/{total_results}: {title[:50]}...")

            except Exception as e:
                print(f"Error processing publication item {index}: {e}")
```

Fig 4: scrape_publication_data() code implementation

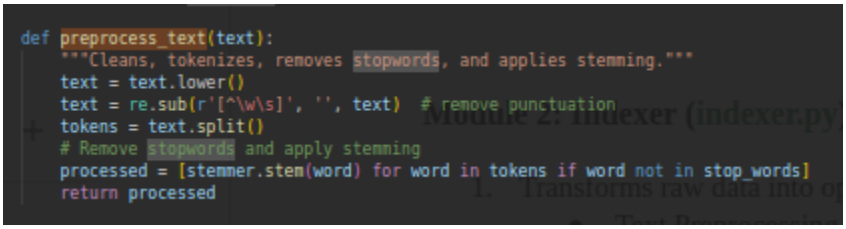
- scrape_author_info() follows detail pages to collect complete author name and profile_url

```python
def scrape_author_info(pub_url, driver, author_hrefs):      You, 2 days ago • Uncommitted changes
    """Scrapes author full names and matches profile URLs using last name + initials."""
    # Check if URL is allowed by robots.txt
    if not is_url_allowed(pub_url):
        print(f"Skipping author info scraping - URL disallowed by robots.txt")
        return []


    try:
        print(f"Navigating to publication page: {pub_url}")
        driver.get(pub_url)

        # 1️ Get full names from cite-author using Selenium
        full_names = []
        try:
            # Wait for the cite-author section to load
            WebDriverWait(driver, 10).until(
                EC.presence_of_element_located((By.ID, "cite-author"))
            )

            # author_div = driver.find_element(By.ID, "cite-author")
            rendering_elements = driver.find_element(By.CLASS_NAME, "rendering_researchoutput")
            print(f"redering_elements found:{rendering_elements}")
            if rendering_elements:
                text = rendering_elements.text.strip()

                print(f"Raw author text: {text}")

                if "/" in text:
                    authors_text = text.split("/")[0].strip()
                else:
                    authors_text = text.strip()

                full_names = [a.strip(" .") for a in authors_text.split(",") if a.strip()]
                print(f"Found {len(full_names)} authors: {full_names}")
            else:
                print("No rendering element found in cite-author")

        except TimeoutException:
            print("Timeout waiting for cite-author section")
            return []
        except NoSuchElementException:
            print("cite-author section not found")
            return []

        # 2️ Get short names + profile URLs from <a rel="Person"> using Selenium
        profile_links = {}
        try:
            # Wait a bit for all elements to load
            time.sleep(2)
            person_links = driver.find_elements(By.CSS_SELECTOR, "a.link.person[rel='Person']")
            print(f"Found {len(person_links)} person links")

            for i, link in enumerate(person_links):
                try:
                    short_name = link.text.strip()   # e.g. "Lis, P."
                    url = link.get_attribute("href")

                    if url and not url.startswith("http"):
                        url = urljoin(pub_url, url)

                    if short_name and url:
                        profile_links[short_name] = url
                        print(f"  Profile {i+1}: {short_name} -> {url}")

                except Exception as e:
                    print(f"Error processing person link {i}: {e}")
                    continue

        except Exception as e:
            print(f"Error finding person links: {e}")

        # 3️ Helper: convert full name -> "Lastname, F."
        def make_initials(name: str) -> str:
            try:
                if "," not in name:
                    # Handle names without comma (First Last format)
                    parts = name.strip().split()
                    if len(parts) >= 2:
```

Fig 5: scrape_author_info() code implementation

Module 2: Indexer (indexer.py)

1. Transforms raw data into optimized search structures for efficient retrieval.
    - Text Preprocessing (preprocess_text())



```
def preprocess_text(text):
    """Cleans, tokenizes, removes stopwords, and applies stemming."""
    text = text.lower()
    text = re.sub(r'[^\w\s]', '', text)  # remove punctuation
    tokens = text.split()
    # Remove stopwords and apply stemming
    processed = [stemmer.stem(word) for word in tokens if word not in stop_words]
    return processed
```

Fig 6: preprocess_text*() code implementation*

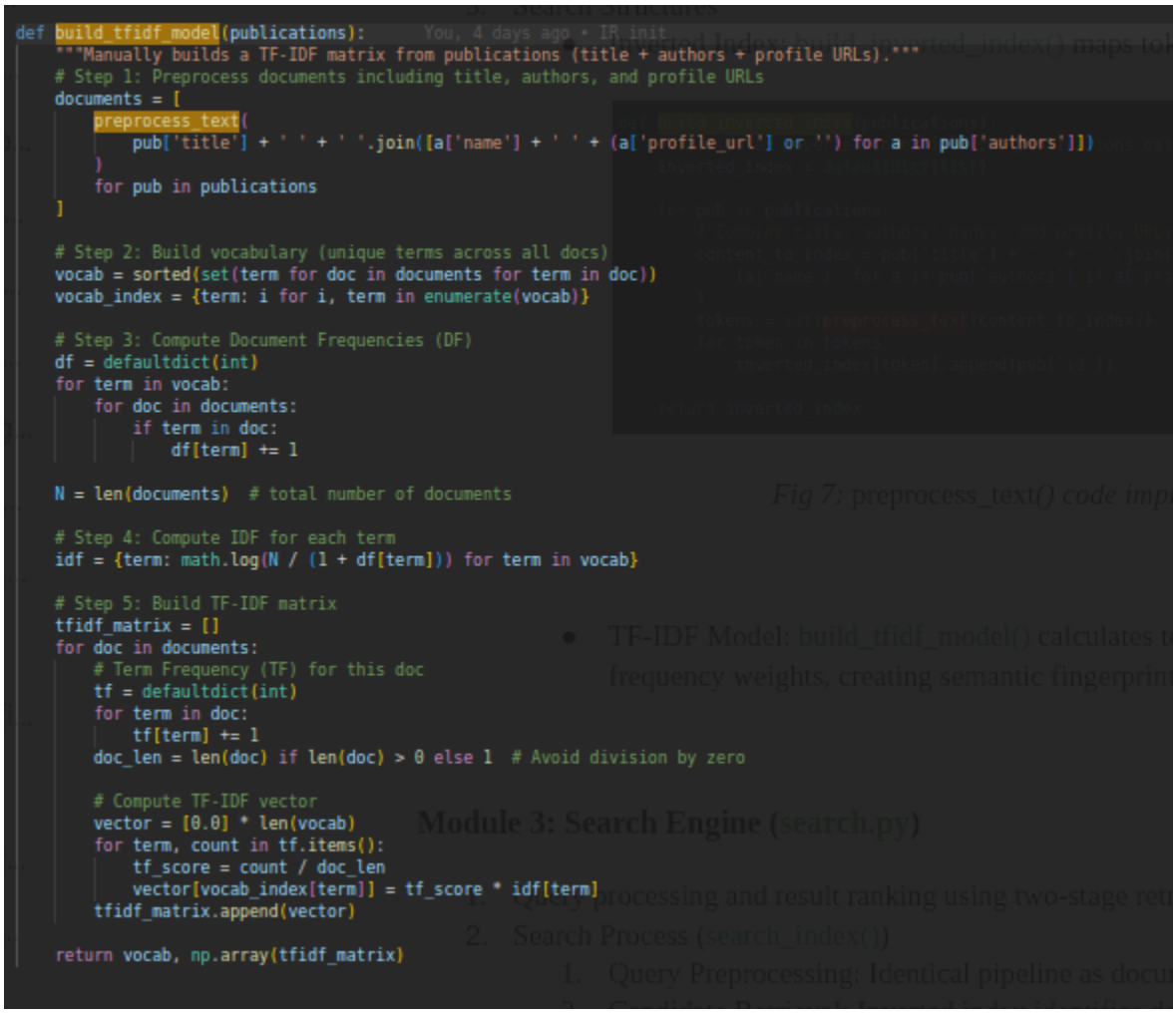
2. Applied identically to documents and queries:
    - Lowercase conversion → punctuation removal → tokenization → stopword elimination → Porter stemming

3. Search Structures
    - Inverted Index: build_inverted_index() maps tokens to document IDs for instant lookups

```
def build_inverted_index(publications):
    """Builds an inverted index from the publications data (title + authors + profile URLs)."""
    inverted_index = defaultdict(list)

    for pub in publications:
        # Combine title, authors' names, and profile URLs into a single string
        content_to_index = pub['title'] + ' ' + ' '.join(
            [a['name']  for a in pub['authors'] if a['profile_url']]
        )
        tokens = set(preprocess_text(content_to_index))  # tokenize, remove stopwords, stem
        for token in tokens:
            inverted_index[token].append(pub['id'])

    return inverted_index
```

Fig 7: preprocess_text*() code implementation*

- TF-IDF Model: build_tfidf_model() calculates term frequency × inverse document frequency weights, creating semantic fingerprints for relevance ranking



```python
def build_tfidf_model(publications):      You, 4 days ago • IR init
    """Manually builds a TF-IDF matrix from publications (title + authors + profile URLs)."""
    # Step 1: Preprocess documents including title, authors, and profile URLs
    documents = [
        preprocess_text(
            pub['title'] + ' ' + ' '.join([a['name'] + ' ' + (a['profile_url'] or '') for a in pub['authors']])
        )
        for pub in publications
    ]

    # Step 2: Build vocabulary (unique terms across all docs)
    vocab = sorted(set(term for doc in documents for term in doc))
    vocab_index = {term: i for i, term in enumerate(vocab)}

    # Step 3: Compute Document Frequencies (DF)
    df = defaultdict(int)
    for term in vocab:
        for doc in documents:
            if term in doc:
                df[term] += 1

    N = len(documents)  # total number of documents

    # Step 4: Compute IDF for each term
    idf = {term: math.log(N / (1 + df[term])) for term in vocab}

    # Step 5: Build TF-IDF matrix
    tfidf_matrix = []
    for doc in documents:
        # Term Frequency (TF) for this doc
        tf = defaultdict(int)
        for term in doc:
            tf[term] += 1
        doc_len = len(doc) if len(doc) > 0 else 1  # Avoid division by zero

        # Compute TF-IDF vector
        vector = [0.0] * len(vocab)
        for term, count in tf.items():
            tf_score = count / doc_len
            vector[vocab_index[term]] = tf_score * idf[term]
        tfidf_matrix.append(vector)

    return vocab, np.array(tfidf_matrix)
```

Fig 8: build_tfidf_model*() code implementation*

Module 3: Search Engine (search.py)

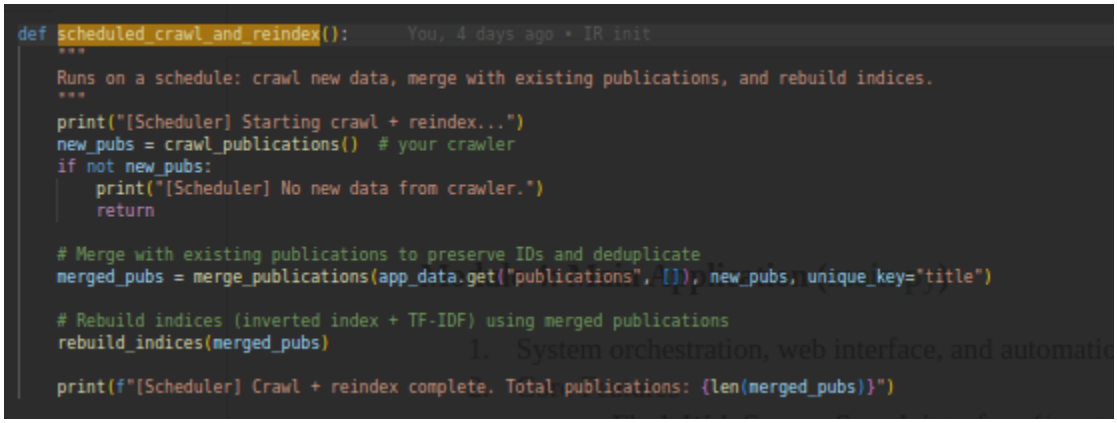
1. Query processing and result ranking using two-stage retrieval.
2. Search Process (search_index())
    1. Query Preprocessing: Identical pipeline as document indexing
    2. Candidate Retrieval: Inverted index identifies documents containing query terms
    3. Relevance Ranking: Cosine similarity between query and document TF-IDF vectors, returning top 10 results

```python
def search_index(query, inverted_index, publications, vocab, tfidf_matrix):
    """Search publications using inverted index and TF-IDF scoring."""
    # Preprocess the query using the same preprocessing function
    query_terms = preprocess_text(query)

    if not query_terms:
        return []

    # Find candidate documents using inverted index
    candidate_ids = set()
    for term in query_terms:
        if term in inverted_index:
            candidate_ids.update(inverted_index[term])

    if not candidate_ids:
        return []

    # Create a mapping from publication ID to matrix index
    id_to_index = {pub['id']: i for i, pub in enumerate(publications)}

    # Get matrix indices for candidate documents
    candidate_matrix_indices = []
    valid_candidate_ids = []

    for pub_id in candidate_ids:
        if pub_id in id_to_index:
            candidate_matrix_indices.append(id_to_index[pub_id])
            valid_candidate_ids.append(pub_id)

    if not candidate_matrix_indices:
        return []

    # Convert to numpy array for proper indexing
    candidate_matrix_indices = np.array(candidate_matrix_indices)

    # Get TF-IDF vectors for candidate documents
    candidate_tfidf_matrix = []
    for idx in candidate_matrix_indices:
        if 0 <= idx < len(tfidf_matrix):
            candidate_tfidf_matrix.append(tfidf_matrix[idx])

    # Create query vector
    vocab_index = {term: i for i, term in enumerate(vocab)}
    query_vector = np.zeros(len(vocab))

    # Simple query term weighting (could be improved with TF-IDF)
    for term in query_terms:
        if term in vocab_index:
            query_vector[vocab_index[term]] = 1.0

    # Compute similarities
    similarities = []
    for i, doc_vector in enumerate(candidate_tfidf_matrix):
        similarity = cosine_similarity(query_vector, doc_vector)
        similarities.append((valid_candidate_ids[i], similarity))

    # Sort by similarity (descending)
    similarities.sort(key=lambda x: x[1], reverse=True)

    # Return top results with publication details
    results = []
    for pub_id, similarity in similarities[:10]:  # Top 10 results
        pub = next((p for p in publications if p['id'] == pub_id), None)
        if pub and similarity > 0:  # Only include results with positive similarity
            results.append({
                'id': pub['id'],
                'title': pub['title'],
                'authors': pub['authors'],
                'similarity': similarity,
                'publication_url': pub['publication_url']
            })

    return results
```

Fig 9: search_index*() code implementation*

Module 4: Main Application (main.py)

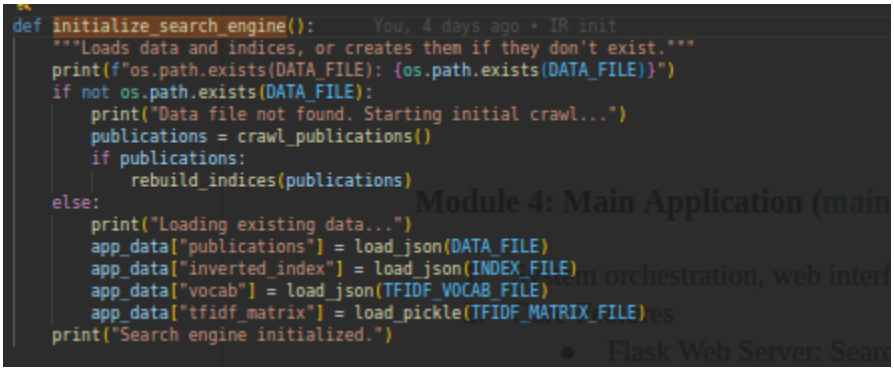
1. System orchestration, web interface, and automation.
2. Core Features
    - Flask Web Server: Search interface (/ route) and JSON API (/search endpoint)
    - Automated Updates: APScheduler runs scheduled_crawl_and_reindex() weekly (Sundays 3 AM)



```
def scheduled_crawl_and_reindex():      You, 4 days ago • IR init
    """
    Runs on a schedule: crawl new data, merge with existing publications, and rebuild indices.
    """
    print("[Scheduler] Starting crawl + reindex...")
    new_pubs = crawl_publications()  # your crawler
    if not new_pubs:
        print("[Scheduler] No new data from crawler.")
        return

    # Merge with existing publications to preserve IDs and deduplicate
    merged_pubs = merge_publications(app_data.get("publications", []), new_pubs, unique_key="title")

    # Rebuild indices (inverted index + TF-IDF) using merged publications
    rebuild_indices(merged_pubs)

    print(f"[Scheduler] Crawl + reindex complete. Total publications: {len(merged_pubs)}")
```

Fig 10: scheduled_crawl_and_reindex() code implementation

- Data Persistence:
    - First run: initialize_search_engine() performs full crawl and indexing

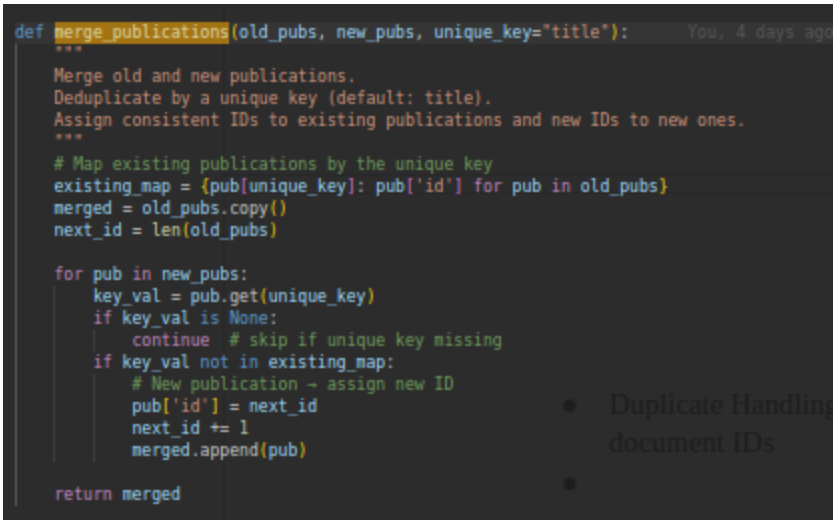


```
def initialize_search_engine():      You, 4 days ago • IR init
    """Loads data and indices, or creates them if they don't exist."""
    print(f"os.path.exists(DATA_FILE): {os.path.exists(DATA_FILE)}")
    if not os.path.exists(DATA_FILE):
        print("Data file not found. Starting initial crawl...")
        publications = crawl_publications()
        if publications:
            rebuild_indices(publications)
    else:
        print("Loading existing data...")
        app_data["publications"] = load_json(DATA_FILE)
        app_data["inverted_index"] = load_json(INDEX_FILE)
        app_data["vocab"] = load_json(TFIDF_VOCAB_FILE)
        app_data["tfidf_matrix"] = load_pickle(TFIDF_MATRIX_FILE)
    print("Search engine initialized.")
```

Fig 11: initialize_search_engine() code implementation

    - Subsequent runs: Load pre-built indices from disk for instant startup

- Duplicate Handling: merge_publications() integrates new data while preserving document IDs

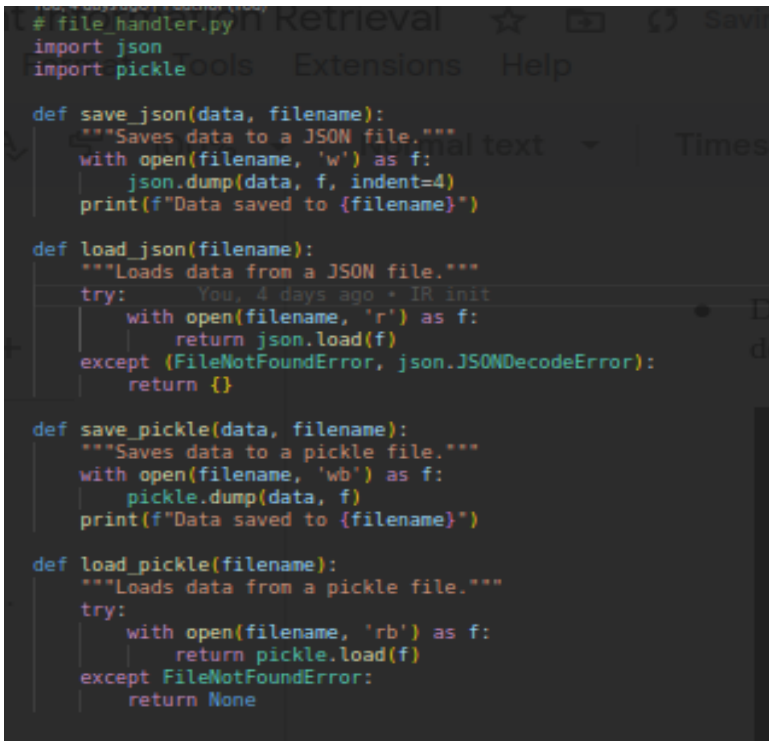


```python
def merge_publications(old_pubs, new_pubs, unique_key="title"):
    """
    Merge old and new publications.
    Deduplicate by a unique key (default: title).
    Assign consistent IDs to existing publications and new IDs to new ones.
    """
    # Map existing publications by the unique key
    existing_map = {pub[unique_key]: pub['id'] for pub in old_pubs}
    merged = old_pubs.copy()
    next_id = len(old_pubs)

    for pub in new_pubs:
        key_val = pub.get(unique_key)
        if key_val is None:
            continue  # skip if unique key missing
        if key_val not in existing_map:
            # New publication → assign new ID
            pub['id'] = next_id
            next_id += 1
            merged.append(pub)

    return merged
```

Fig 12: merge_publications() code implementation

Module 5: Data Management(fileHandler.py)

All file operations handled by fileHandler.py using JSON and Pickle formats for reliable storage and retrieval.



```python
# file_handler.py
import json
import pickle

def save_json(data, filename):
    """Saves data to a JSON file."""
    with open(filename, 'w') as f:
        json.dump(data, f, indent=4)
    print(f"Data saved to {filename}")

def load_json(filename):
    """Loads data from a JSON file."""
    try:
        with open(filename, 'r') as f:
            return json.load(f)
    except (FileNotFoundError, json.JSONDecodeError):
        return {}

def save_pickle(data, filename):
    """Saves data to a pickle file."""
    with open(filename, 'wb') as f:
        pickle.dump(data, f)
    print(f"Data saved to {filename}")

def load_pickle(filename):
    """Loads data from a pickle file."""
    try:
        with open(filename, 'rb') as f:
            return pickle.load(f)
    except FileNotFoundError:
        return None
```

Fig 13: fileHandler.py code implementation

1.4. Workflow Image

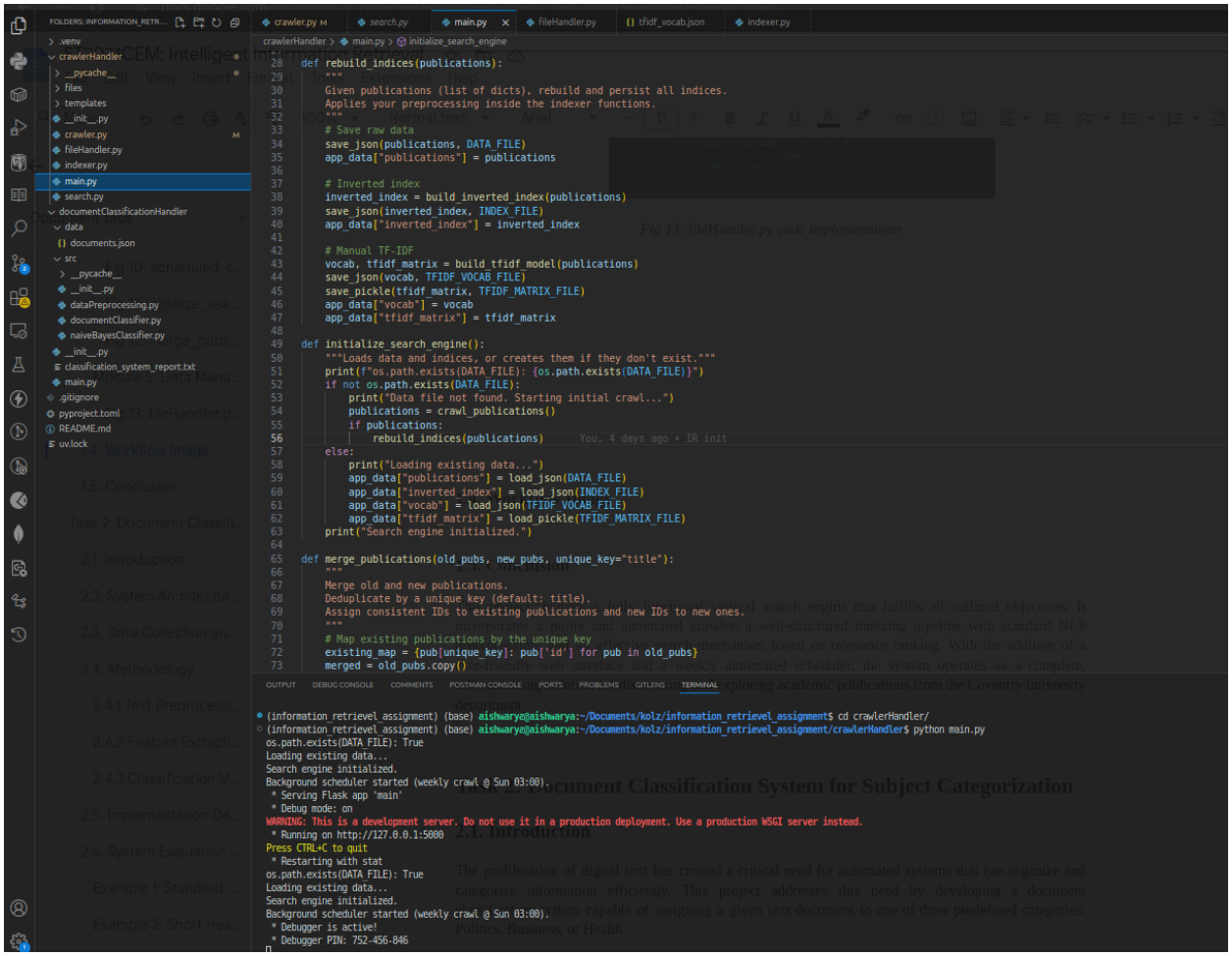


Fig 14: Local Server Up for crawler

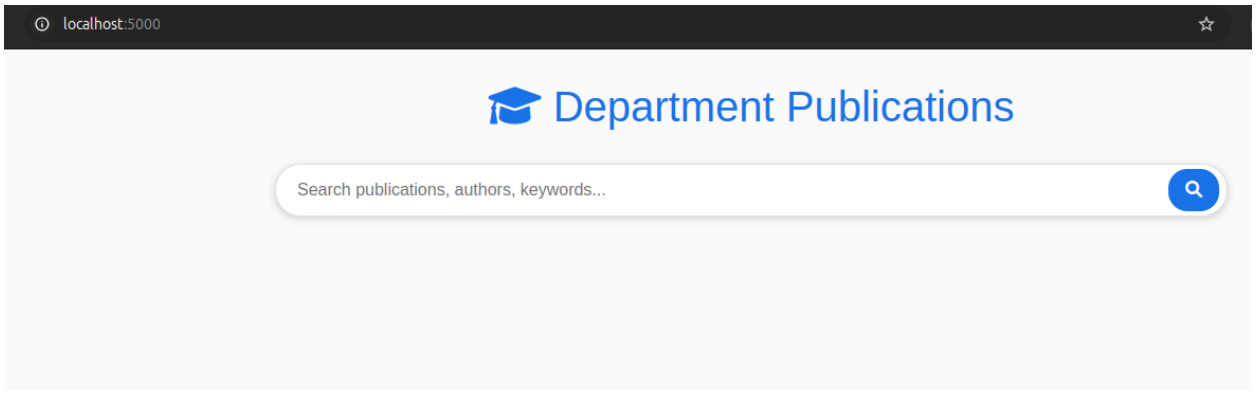


Fig 15: Crawler UI

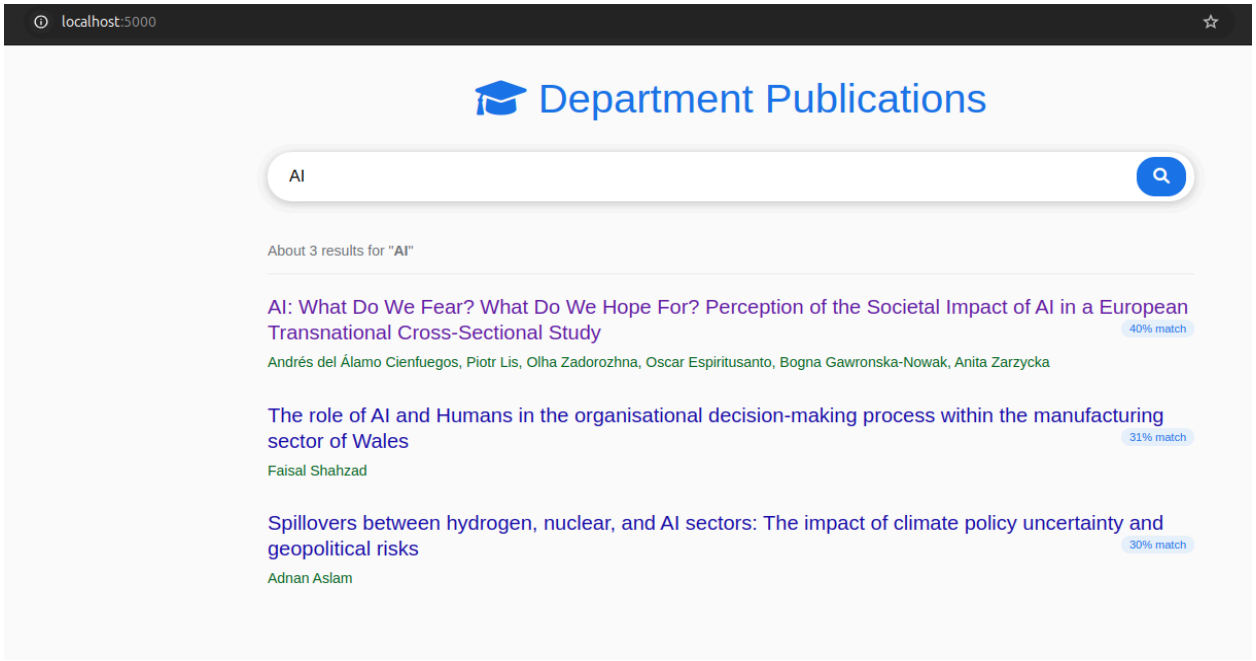


Fig 16: Search for publications published by Coventry University's School of Economics, Finance, and Accounting Department

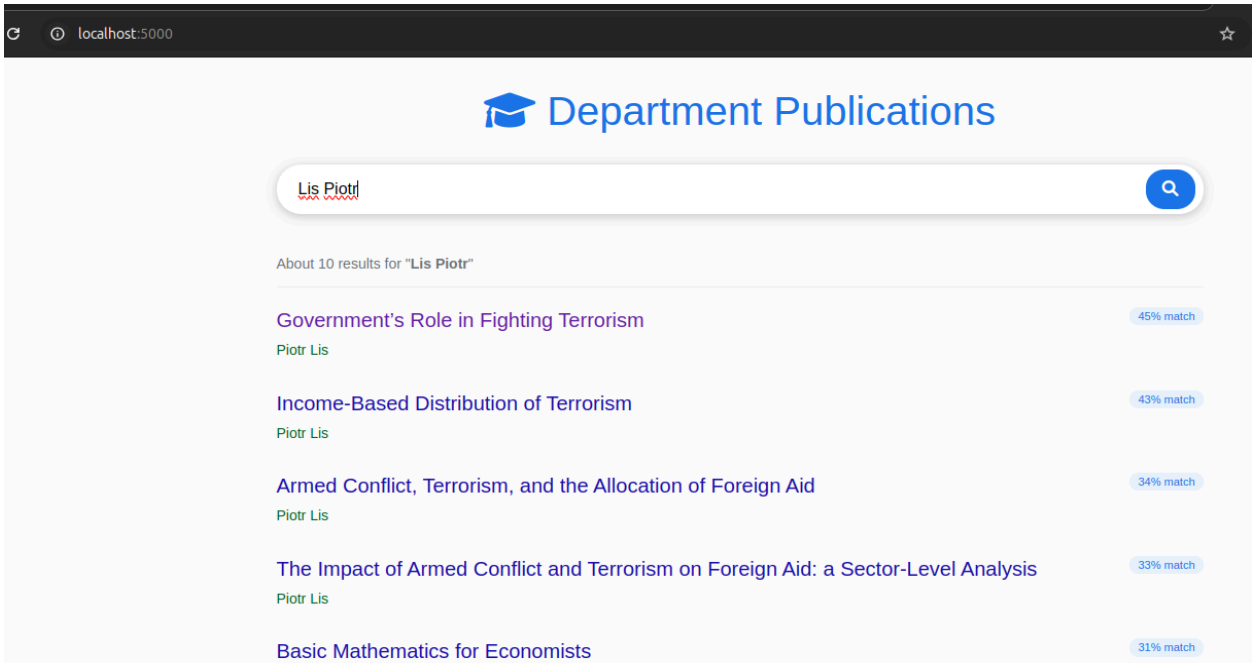


Fig 17: Search for publications published by Authors of Coventry University's School of Economics, Finance, and Accounting Department

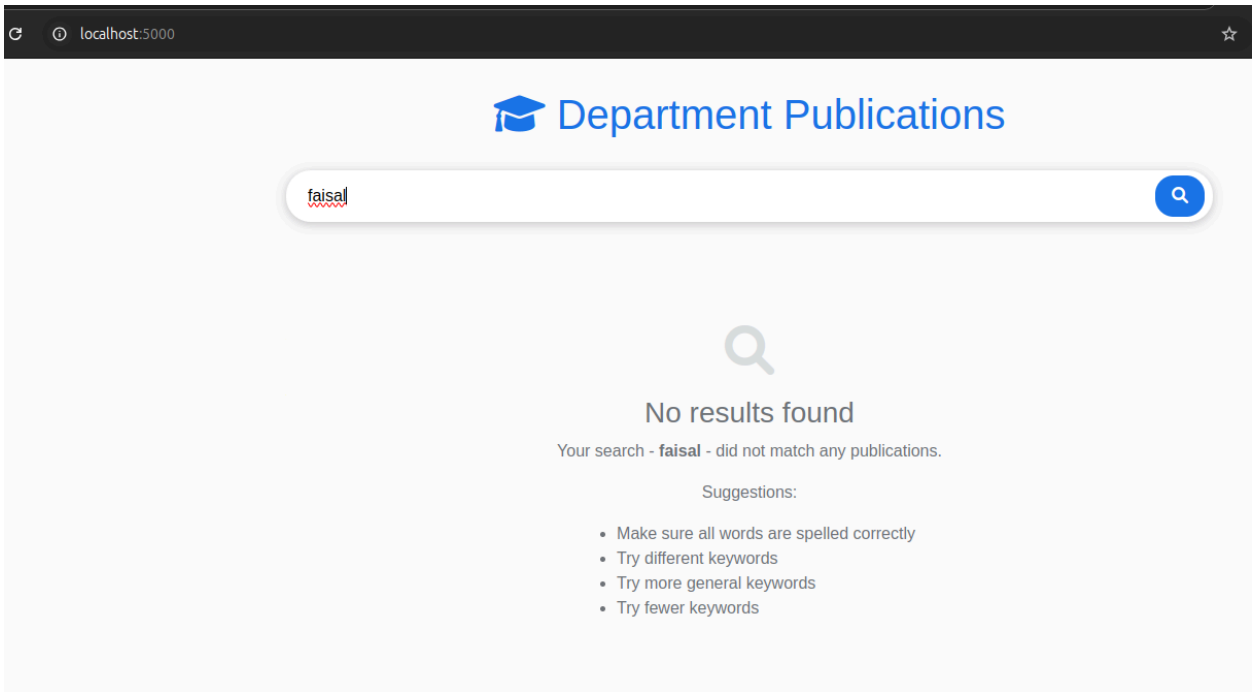


Fig 18: Search for publications published by Authors of Coventry University's other department except School of Economics, Finance, and Accounting

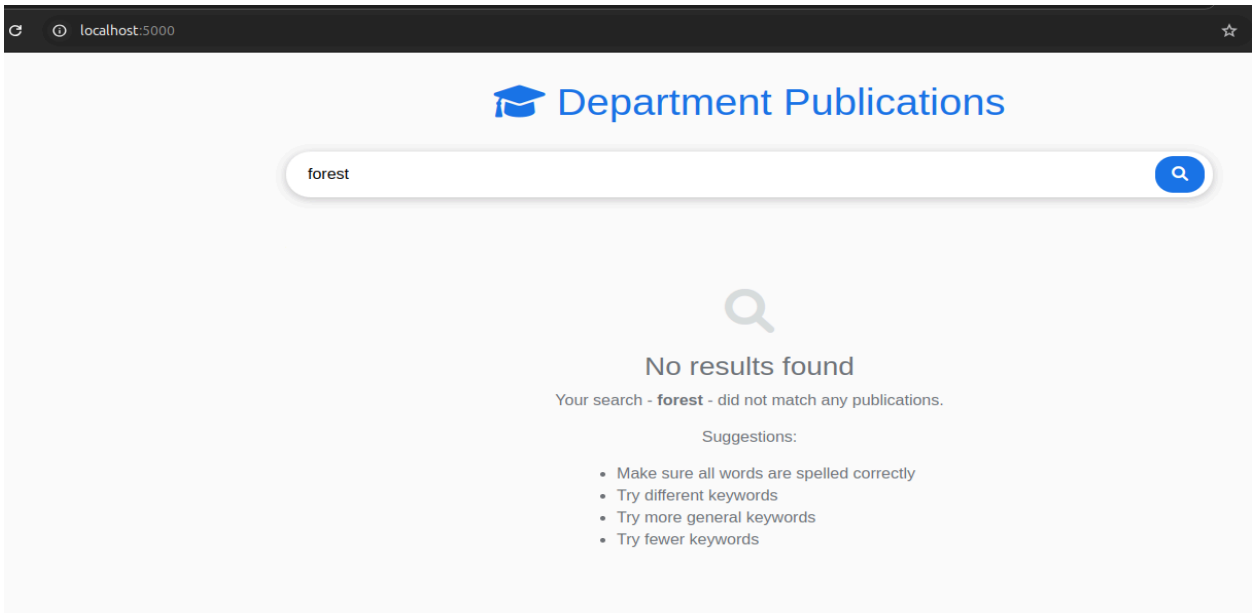


Fig 19: Search for publications published Coventry University's other department except School of Economics, Finance, and Accounting

1.5. Conclusion

This project delivers a fully functional vertical search engine that fulfills all outlined objectives. It incorporates a polite and automated crawler, a well-structured indexing pipeline with standard NLP preprocessing, and an effective search mechanism based on relevance ranking. With the addition of a user-friendly web interface and a weekly automated scheduler, the system operates as a complete, self-sustaining solution for discovering and exploring academic publications from the Coventry university department.

Task 2: Document Classification System for Subject Categorization

2.1. Project Overview

This project develops an automated text categorization system that classifies documents into three categories: Politics, Business, or Health. Built with a Multinomial Naïve Bayes classifier, the system implements a complete pipeline from data preprocessing to prediction using Python, NLTK, and scikit-learn.

The system addresses the growing need for efficient organization of digital text content through automated classification, providing reliable categorization with demonstrated performance across diverse document types.

2.2. System Architecture and Design

The system uses modular architecture with distinct components for maintainability and efficiency. The workflow follows a five-stage pipeline:

1. **Data Loading**: Raw text documents loaded from JSON file
2. **Preprocessing**: Text cleaning and normalization for feature extraction
3. **Feature Extraction**: Text converted to TF-IDF numerical vectors
4. **Model Training**: Multinomial Naïve Bayes classifier trained on vectors and labels
5. **Prediction**: Trained model classifies new documents

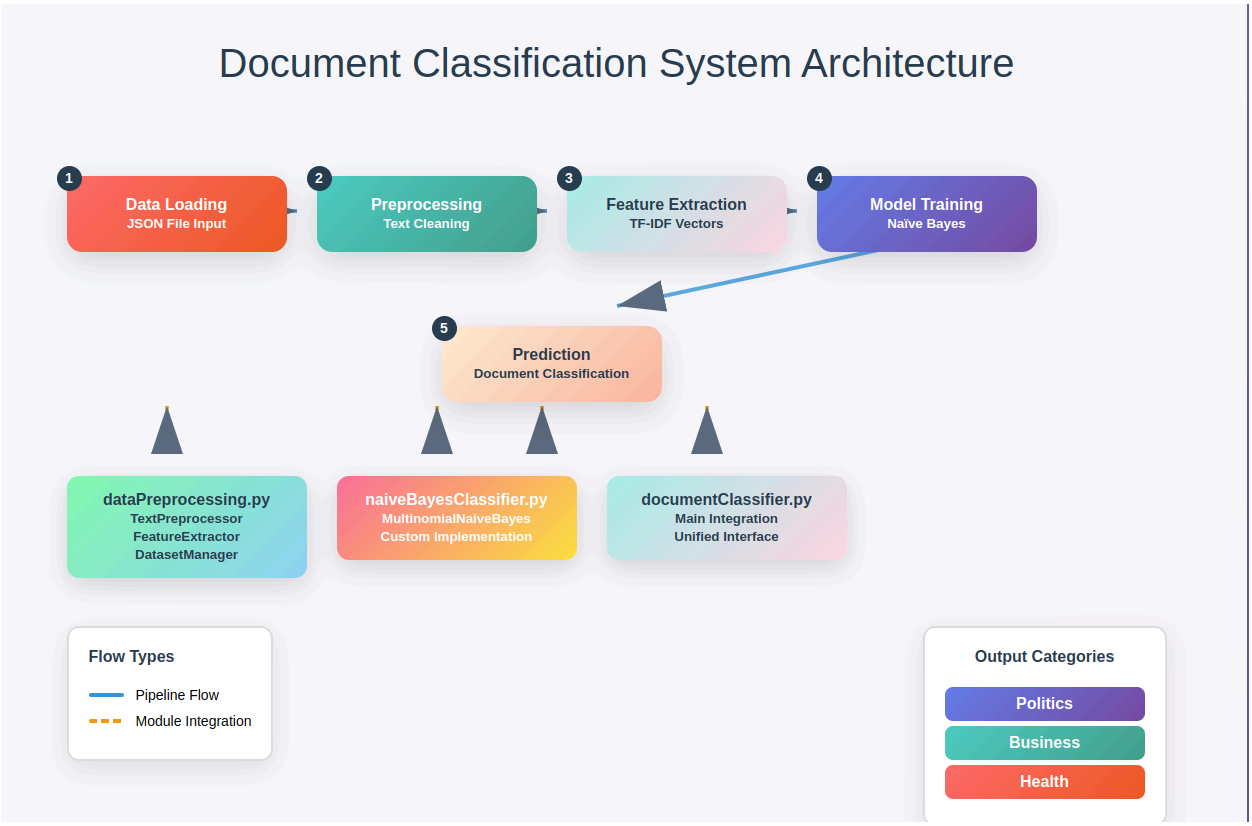


Fig 1: Document Classification System Architecture

2.3 Implementation Structure:

- **DataPreprocessing.py**: TextPreprocessor, FeatureExtractor, and DatasetManager classes handle data loading and transformation
- **NaiveBayesClassifier.py**: Custom MultinomialNaiveBayes implementation with core mathematical logic
- **DocumentClassifier.py**: Main DocumentClassifier class integrating all components with unified interface for training, prediction, and model persistence

2.4. Data Collection and Preparation

A dataset of over 100 documents was manually collected from the different News website, covering the three target categories.

- Categories: Politics, Business, Health.
- Format: The collected documents were stored in a JSON file, with each entry being an object containing two keys: "text" (the document content) and "category" (the corresponding label).

This approach ensures a clean and structured dataset for training and evaluation.

2.5. Methodology

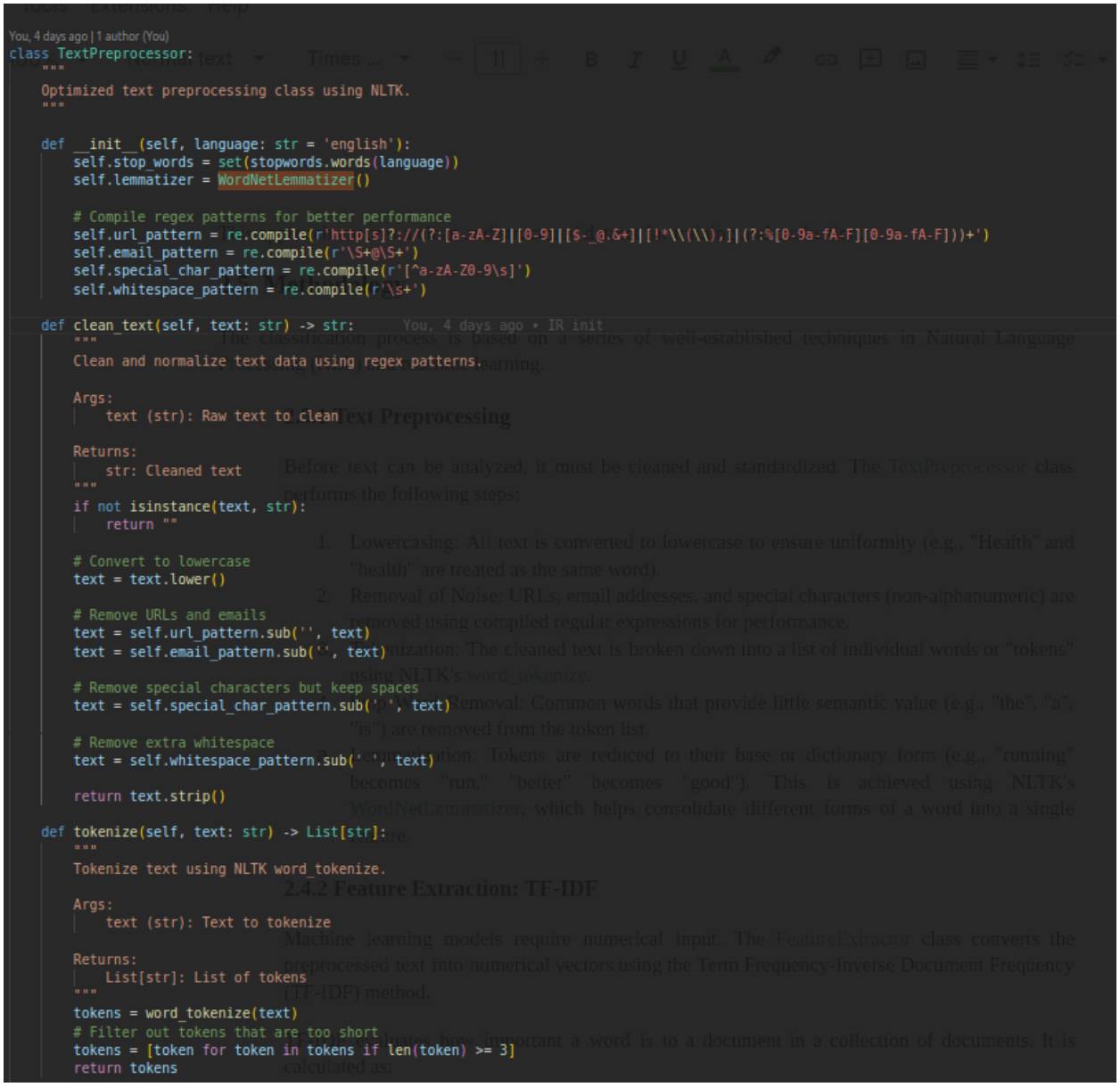
The classification process is based on a series of well-established techniques in Natural Language Processing (NLP) and machine learning.

2.5.1 Text Preprocessing

Before text can be analyzed, it must be cleaned and standardized. The TextPreprocessor class performs the following steps:

1. Lowercasing: All text is converted to lowercase to ensure uniformity (e.g., "Health" and "health" are treated as the same word).
2. Removal of Noise: URLs, email addresses, and special characters (non-alphanumeric) are removed using compiled regular expressions for performance.
3. Tokenization: The cleaned text is broken down into a list of individual words or "tokens" using NLTK's word_tokenize.
4. Stop Word Removal: Common words that provide little semantic value (e.g., "the", "a", "is") are removed from the token list.

a. Lemmatization: Tokens are reduced to their base or dictionary form (e.g., "running" becomes "run," "better" becomes "good"). This is achieved using NLTK's WordNetLemmatizer, which helps consolidate different forms of a word into a single feature.



```python
You, 4 days ago | 1 author (You)
class TextPreprocessor:
    """
    Optimized text preprocessing class using NLTK.
    """

    def __init__(self, language: str = 'english'):
        self.stop_words = set(stopwords.words(language))
        self.lemmatizer = WordNetLemmatizer()

        # Compile regex patterns for better performance
        self.url_pattern = re.compile(r'http[s]?://(?:[a-zA-Z]|[0-9]|[$-_@.&+]|[!*\\(\\),]|(?:%[0-9a-fA-F][0-9a-fA-F]))+')
        self.email_pattern = re.compile(r'\S+@\S+')
        self.special_char_pattern = re.compile(r'[^a-zA-Z0-9\s]')
        self.whitespace_pattern = re.compile(r'\s+')

    def clean_text(self, text: str) -> str:      You, 4 days ago • IR init
        """
        Clean and normalize text data using regex patterns.

        Args:
            text (str): Raw text to clean

        Returns:
            str: Cleaned text
        """
        if not isinstance(text, str):
            return ""

        # Convert to lowercase
        text = text.lower()

        # Remove URLs and emails
        text = self.url_pattern.sub('', text)
        text = self.email_pattern.sub('', text)

        # Remove special characters but keep spaces
        text = self.special_char_pattern.sub('', text)

        # Remove extra whitespace
        text = self.whitespace_pattern.sub(' ', text)

        return text.strip()

    def tokenize(self, text: str) -> List[str]:
        """
        Tokenize text using NLTK word_tokenize.

        Args:
            text (str): Text to tokenize

        Returns:
            List[str]: List of tokens
        """
        tokens = word_tokenize(text)
        # Filter out tokens that are too short
        tokens = [token for token in tokens if len(token) >= 3]
        return tokens
```

Fig 2: TextPreprocessor class code snippet

2.5.2 Feature Extraction: TF-IDF

Machine learning models require numerical input. The FeatureExtractor class converts the preprocessed text into numerical vectors using the Term Frequency-Inverse Document Frequency (TF-IDF) method.

TF-IDF evaluates how important a word is to a document in a collection of documents. It is calculated as:

$$\text{TF-IDF}(t,d,D) = \text{TF}(t,d) \times \text{IDF}(t,D)$$

Where:

- TF(t, d): Term Frequency is the number of times the term t appears in document d.
- IDF(t, D): Inverse Document Frequency measures how much information the word provides. It is calculated as the logarithm of the total number of documents D divided by the number of documents containing the term t.

This technique gives higher weight to words that are frequent in a specific document but rare across all documents, making them good indicators of the document's topic. Our implementation uses scikit-learn's TfidfVectorizer with n-grams of range (1, 2) to capture both single words and two-word phrases.

```python
class FeatureExtractor:
    """
    Feature extraction using TF-IDF vectorization.
    """

    def __init__(self, max_features: int = 5000, ngram_range: Tuple[int, int] = (1, 2)):
        """
        Initialize feature extractor.

        Args:
            max_features (int): Maximum number of features
            ngram_range (Tuple[int, int]): N-gram range for TF-IDF
        """
        self.vectorizer = TfidfVectorizer(
            max_features=max_features,
            ngram_range=ngram_range,
            min_df=2,
            max_df=0.8,
            lowercase=False,  # Already handled in preprocessing
            analyzer='word'
        )
        self.is_fitted = False

    def fit_transform(self, texts: List[str]) -> np.ndarray:
        """
        Fit vectorizer and transform texts.

        Args:
            texts (List[str]): List of preprocessed texts

        Returns:
            np.ndarray: TF-IDF feature matrix
        """
        features = self.vectorizer.fit_transform(texts)
        self.is_fitted = True
        return features.toarray() # type: ignore

    def transform(self, texts: List[str]) -> np.ndarray:
        """
        Transform texts using fitted vectorizer.

        Args:
            texts (List[str]): List of preprocessed texts

        Returns:
            np.ndarray: TF-IDF feature matrix
        """
        if not self.is_fitted:
            raise ValueError("Vectorizer must be fitted before transform")

        features = self.vectorizer.transform(texts)
        return features.toarray() # type: ignore

    def get_feature_names(self) -> List[str]:
        """
        Get feature names from fitted vectorizer.

        Returns:
            List[str]: List of feature names
        """
        if not self.is_fitted:
            raise ValueError("Vectorizer must be fitted first")
```

Fig 3: FeatureExtractor *class code snippet*

2.5.3 Classification Model: Multinomial Naïve Bayes

The core of our system is the Multinomial Naïve Bayes classifier, a probabilistic algorithm based on Bayes' Theorem:

$$P(C|D)=P(D)P(D|C)\cdot P(C)$$

Where:

- $P(C|D)$ is the probability of a document D belonging to class C.
- $P(C)$ is the prior probability of class C.
- $P(D|C)$ is the likelihood of observing document D given it belongs to class C.
- $P(D)$ is the probability of observing document D.

The "naïve" assumption of this model is that all features (words) are independent of each other, which simplifies the calculation of $P(D|C)$.

Our implementation in naiveBayesClassifier.py includes two critical enhancements:

- Laplace (Additive) Smoothing (alpha=1.0): This prevents issues with words that appear in the test set but not in the training set for a particular class. It avoids zero probabilities, which would otherwise nullify the entire calculation.
- Log Probabilities: To prevent numerical underflow (a common issue when multiplying many small probabilities), all calculations are performed in log space.

```python
class MultinomialNaiveBayes:
    """
    Multinomial Naive Bayes classifier optimized for text classification.
    Uses log probabilities and vectorized operations for efficiency and numerical stability.
    """

    def __init__(self, alpha: float = 1.0):
        """
        Initialize the Naive Bayes classifier.

        Args:
            alpha (float): Laplace smoothing parameter (default: 1.0)
                           Higher values = more smoothing, less overfitting
                           Lower values = less smoothing, may overfit on small datasets
        """
        self.alpha = alpha
        self.class_log_priors = None
        self.feature_log_probs = None
        self.classes = None
        self.class_names = None
        self.n_features = 0
        self.is_fitted = False

        # Store label mappings for string labels
        self.label_to_idx = None
        self.idx_to_label = None

    def fit(self, X: np.ndarray, y: Union[np.ndarray, List[str]]):
        """
        Train the Naive Bayes classifier on feature matrix and labels.

        Args:
            X (np.ndarray): Feature matrix of shape (n_samples, n_features)
                            Each row represents a document, each column a feature
            y (Union[np.ndarray, List[str]]): Target labels
                                              Can be string labels or integer indices
        """

        print(f"fit called")
        # Handle string labels by converting to indices
        if isinstance(y[0], str):
            unique_labels = sorted(list(set(y))) # type: ignore
            self.label_to_idx = {label: idx for idx, label in enumerate(unique_labels)}
            self.idx_to_label = {idx: label for label, idx in self.label_to_idx.items()}
            y_encoded = np.array([self.label_to_idx[label] for label in y])
            self.class_names = unique_labels
        else:
            y_encoded = np.array(y)
            self.classes = np.unique(y_encoded)
            self.class_names = [f"Class_{i}" for i in self.classes]

        self.classes = np.unique(y_encoded)
        n_classes = len(self.classes)
        n_samples, self.n_features = X.shape

        # Calculate class priors using log probabilities for numerical stability
        class_counts = np.bincount(y_encoded)
        self.class_log_priors = np.log(class_counts / n_samples)

        # Initialize feature probabilities matrix (n_classes x n_features)
        self.feature_log_probs = np.zeros((n_classes, self.n_features))

        # Calculate feature likelihoods for each class using vectorized operations
        for i, class_idx in enumerate(self.classes):
            # Get all samples belonging to this class
            class_mask = (y_encoded == class_idx)
```

Fig 4: MultinomialNaiveBayes class code snippet

2.6. Implementation Details

The system is implemented using the following key libraries:

- NLTK: For core NLP tasks like tokenization, stop word removal, and lemmatization.
- scikit-learn: For TF-IDF vectorization and data splitting.
- NumPy: For efficient numerical computations in the Naïve Bayes algorithm.

The main workflow, managed by the DocumentClassifier class, is as follows:

1. Training: The train_from_file() method loads the JSON data, orchestrates the preprocessing and feature extraction via DatasetManager, and then calls the fit() method of the MultinomialNaiveBayes classifier.

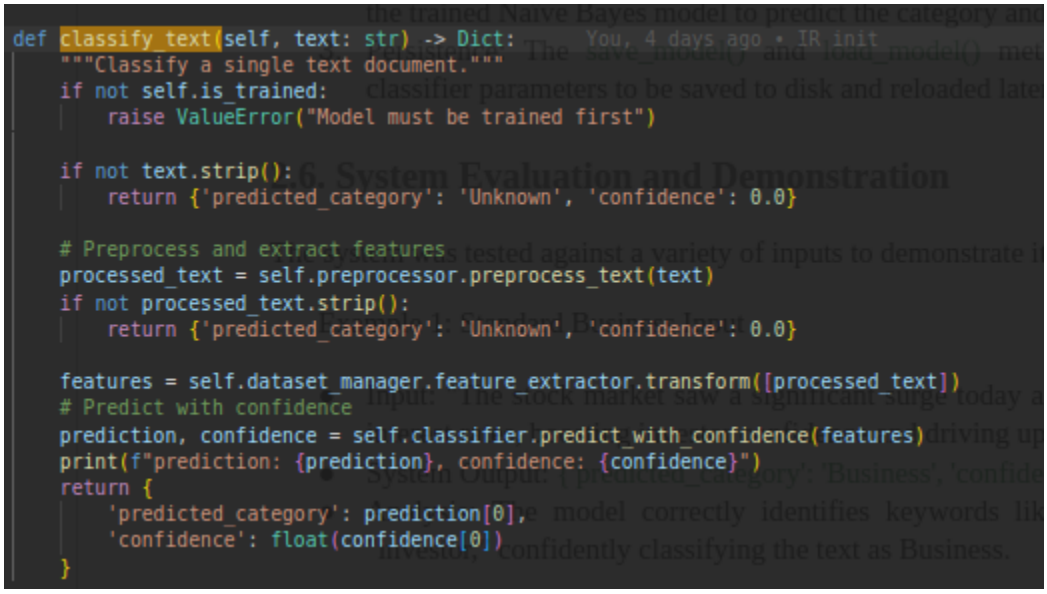
```
def train_from_file(self, filepath: str, test_size: float = 0.2,       You, 4 days
                    random_state: int = 42) -> Dict:
    """Train from JSON file."""
    print(f"filepath: {filepath}")
    documents = self.dataset_manager.load_raw_data(filepath)
    print(f"documents: {len(documents)}")
    return self.train_from_documents(documents, test_size, random_state)
```

Fig 5: train_from_file() code implementation

```python
def fit(self, X: np.ndarray, y: Union[np.ndarray, List[str]]):      You, 4 days ago • IR init
    """
    Train the Naive Bayes classifier on feature matrix and labels.

    Args:
        X (np.ndarray): Feature matrix of shape (n_samples, n_features)
                        Each row represents a document, each column a feature
        y (Union[np.ndarray, List[str]]): Target labels
                                          Can be string labels or integer indices
    """

    print(f"fit called")
    # Handle string labels by converting to indices
    if isinstance(y[0], str):
        unique_labels = sorted(list(set(y))) # type: ignore
        self.label_to_idx = {label: idx for idx, label in enumerate(unique_labels)}
        self.idx_to_label = {idx: label for label, idx in self.label_to_idx.items()}
        y_encoded = np.array([self.label_to_idx[label] for label in y])
        self.class_names = unique_labels
    else:
        y_encoded = np.array(y)
        self.classes = np.unique(y_encoded)
        self.class_names = [f"Class_{i}" for i in self.classes]

    self.classes = np.unique(y_encoded)
    n_classes = len(self.classes)
    n_samples, self.n_features = X.shape

    # Calculate class priors using log probabilities for numerical stability
    class_counts = np.bincount(y_encoded)
    self.class_log_priors = np.log(class_counts / n_samples)

    # Initialize feature probabilities matrix (n_classes x n_features)
    self.feature_log_probs = np.zeros((n_classes, self.n_features))

    # Calculate feature likelihoods for each class using vectorized operations
    for i, class_idx in enumerate(self.classes):
        # Get all samples belonging to this class
        class_mask = (y_encoded == class_idx)
        class_features = X[class_mask]

        # Sum feature counts for this class across all documents
        feature_counts = np.sum(class_features, axis=0)

        # Apply Laplace smoothing to avoid zero probabilities
        smoothed_counts = feature_counts + self.alpha
        total_count = np.sum(smoothed_counts)

        # Calculate log probabilities for numerical stability
        self.feature_log_probs[i] = np.log(smoothed_counts / total_count)

    self.is_fitted = True
    print(f"Model trained: {n_samples} samples, {n_classes} classes, {self.n_features} features")
```

Fig 6: fit() code implementation

2. Prediction: The classify_text() method takes a new string of text, applies the *same* preprocessing pipeline, transforms it into a TF-IDF vector using the *already fitted* vectorizer, and finally uses the trained Naïve Bayes model to predict the category and a confidence score.



```
def classify_text(self, text: str) -> Dict:
    """Classify a single text document."""
    if not self.is_trained:
        raise ValueError("Model must be trained first")

    if not text.strip():
        return {'predicted_category': 'Unknown', 'confidence': 0.0}

    # Preprocess and extract features
    processed_text = self.preprocessor.preprocess_text(text)
    if not processed_text.strip():
        return {'predicted_category': 'Unknown', 'confidence': 0.0}

    features = self.dataset_manager.feature_extractor.transform([processed_text])
    # Predict with confidence
    prediction, confidence = self.classifier.predict_with_confidence(features)
    print(f"prediction: {prediction}, confidence: {confidence}")
    return {
        'predicted_category': prediction[0],
        'confidence': float(confidence[0])
    }
```

Fig 7: classify_text*() code implementation*

3. Persistence: The save_model() and load_model() methods allow the trained vectorizer and classifier parameters to be saved to disk and reloaded later, avoiding the need for retraining.

```python
def save_model(self, model_dir: str = "models"):
    """Save the trained model."""
    if not self.is_trained:
        raise ValueError("Model must be trained first")

    model_path = Path(model_dir)
    model_path.mkdir(parents=True, exist_ok=True)

    # Save classifier and vectorizer
    self.classifier.save_model(str(model_path / "naive_bayes_model.json"))
    self.dataset_manager.feature_extractor.save_vectorizer(
        str(model_path / "vectorizer.pkl")
    )       You, 4 days ago • IR init

    # Save training results
    with open(model_path / "training_results.json", 'w') as f:
        json.dump(self.training_results, f, indent=2)

    print(f"Model saved to {model_path}/")

def load_model(self, model_dir: str = "models"):
    """Load a trained model."""
    model_path = Path(model_dir)

    if not model_path.exists():
        raise FileNotFoundError(f"Model directory not found: {model_path}")

    # Load classifier and vectorizer
    self.classifier.load_model(str(model_path / "naive_bayes_model.json"))
    self.dataset_manager.feature_extractor.load_vectorizer(
        str(model_path / "vectorizer.pkl")
    )

    # Load training results
    with open(model_path / "training_results.json", 'r') as f:
        self.training_results = json.load(f)

    self.is_trained = True
    print(f"Model loaded from {model_path}/")
```

Fig 8: save_model*() and load_model() code implementation*

2.7. Top 6 Reasons to Use Naive Bayes Over Logistic Regression:

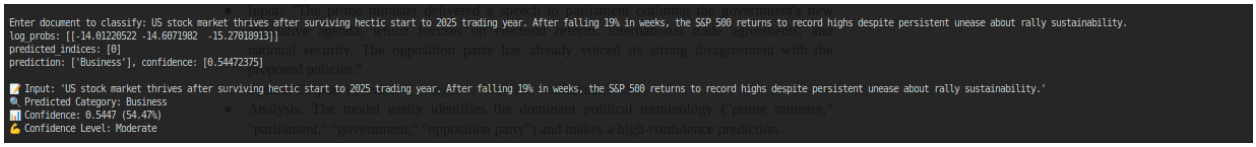
- **Less overfitting** - Simple model with fewer parameters to estimate
- **No minimum sample size requirement** - Works even with very few training examples
- **Stable performance** - Doesn't need large amounts of data for reliable results
- **Fast training** - Quick to train even on tiny datasets
- **Better generalization** - Simple assumptions prevent memorizing small training sets
- **No convergence problems** - Logistic regression may fail to converge with insufficient data

2.8. System Evaluation and Demonstration

The system was tested against a variety of inputs to demonstrate its robustness.

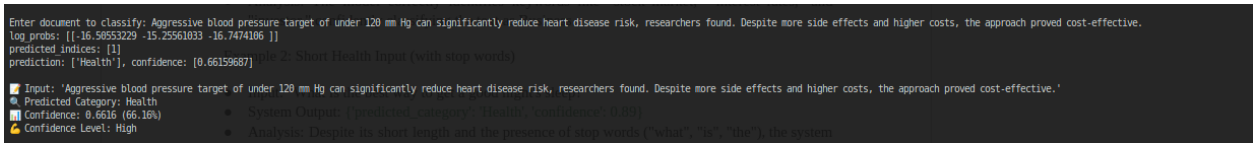
Example 1: Business Input

- Input: "US stock market thrives after surviving hectic start to 2025 trading year. After falling 19% in weeks, the S&P 500 returns to record highs despite persistent unease about rally sustainability."
- System Output:
  - 🔍 Predicted Category: Business
  - 📊 Confidence: 0.5447 (54.47%)
  - 💪 Confidence Level: Moderate



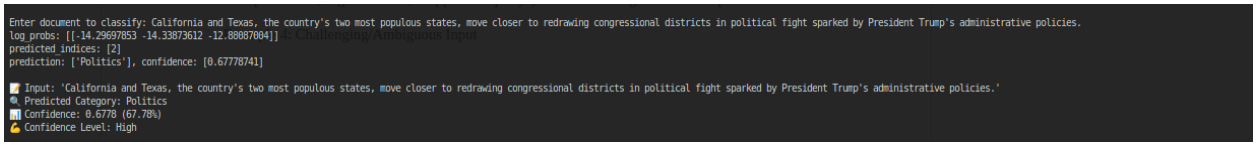
Example 2: Health Input

- Input: "Aggressive blood pressure target of under 120 mm Hg can significantly reduce heart disease risk, researchers found. Despite more side effects and higher costs, the approach proved cost-effective."
- System Output:
  - 🔍 Predicted Category: Health
  - 📊 Confidence: 0.6616 (66.16%)
  - 💪 Confidence Level: High



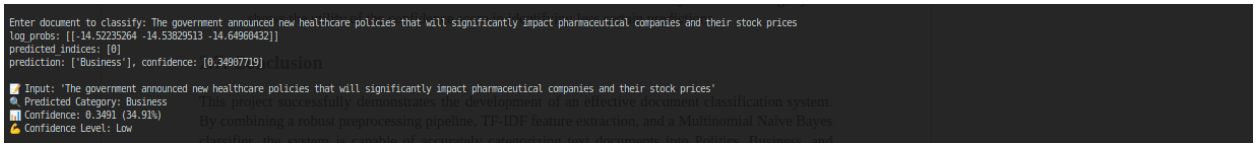
Example 3: Political Input

- Input: "California and Texas, the country's two most populous states, move closer to redrawing congressional districts in political fight sparked by President Trump's administrative policies."
- System Output:
  - 🔍 Predicted Category: Politics
  - 📊 Confidence: 0.6778 (67.78%)
  - 💪 Confidence Level: High



Example 4: Challenging/Ambiguous Input

- Input: "The government announced new healthcare regulations that will significantly impact pharmaceutical companies and their stock prices."
- System Output:
  - 🔍 Predicted Category: Business
  - 📊 Confidence: 0.3491 (34.91%)
  - 💪 Confidence Level: Low
- Analysis: This input contains keywords from all three categories ("government" -> Politics, "healthcare" -> Health, "companies" & "stock prices" -> Business). The model predicts Business but with a lower confidence score. This demonstrates the model's ability to handle ambiguity and shows the utility of the confidence score in identifying less certain predictions.



2.9. Conclusion

This project successfully demonstrates the development of an effective document classification system. By combining a robust preprocessing pipeline, TF-IDF feature extraction, and a Multinomial Naïve Bayes classifier, the system is capable of accurately categorizing text documents into Politics, Business, and Health. The modular design ensures that the system is both maintainable and extensible.

References

Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media, Inc.

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press.

Ramos, J. (2003). "Using TF-IDF to Determine Word Relevance in Document Queries"

Rish, I. (2001). "An empirical study of the naïve Bayes classifier." IJCAI Workshop

Github Repo Link

https://github.com/aishwaryawambule/information_retrieval_agentic

Screen Record Video of Task 1 and Task 2

https://drive.google.com/file/d/1JRhkNgOP-1bwX7NKDZToVxfCvWFPgclY/view?usp=sharing